ex3 converted (1)

ex3

23IT53C – Modern Web Technologies Laboratory Department Of Information Technology Ex.No : 3 CallBack,Promises,Async/Await Programs on javascript Date: AIM:To implement JavaScript programs that demonstrate asynchronous programming concepts using Callbacks, Promises, and Async/Await for executing dependent tasks in sequence with simulated delays. 3.a E-Commerce Order Processing An e-commerce platform has addToCart(product, callback / promise / async) – adds a product to the cart (1 sec delay), placeOrder(order, callback / promise / async) – places the order (2 sec delay), sendConfirmationEmail(order) – sends order confirmation email. Program: const cart = []; const addToCart = (product) => { return new Promise((resolve) => { setTimeout(() => { cart.push(product); console.log(`[Promise] Added to cart: ${product.name}`); resolve([...cart]); }, 1000); }); }; const placeOrder = (order) => { return new Promise((resolve) => { setTimeout(() => { const orderId = 'ORD-' + Math.floor(1000 + Math.random() * 9000); console.log(`[Promise] Order placed! ID: ${orderId}`); resolve({ orderId, items: order }); }, 2000); }); }; function addToCartCb(product, callback) { setTimeout(() => { cart.push(product); console.log(`[Callback] Added to cart: ${product.name}`); callback(null, [...cart]); }, 1000); } function placeOrderCb(order, callback) { setTimeout(() => { const orderId = 'ORD-' + Math.floor(1000 + Math.random() * 9000); Register No.24205038

23IT53C – Modern Web Technologies Laboratory Department Of Information Technology console.log(`[Callback] Order placed! ID: ${orderId}`); callback(null, { orderId, items: order }); }, 2000); } const sendConfirmationEmail = (orderResult) => { console.log(`[Email] Confirmation sent for Order: ${orderResult.orderId}`); }; async function runAsyncAwaitDemo() { console.log("\n--- Starting Async/Await Workflow ---"); const item = { id: 101, name: "Wireless Headphones", price: 99.99 }; const updatedCart = await addToCart(item); const orderDetails = await placeOrder(updatedCart); sendConfirmationEmail(orderDetails); console.log("Async/Await Workflow Complete!"); } function runPromiseChainDemo() { console.log("\n--- Starting Promise Chain Workflow ---"); const item = { id: 102, name: "Mechanical Keyboard", price: 120.00 }; return addToCart(item) .then((updatedCart) => placeOrder(updatedCart)) .then((orderDetails) => { sendConfirmationEmail(orderDetails); console.log("Promise Chain Workflow Complete!"); }); } function runCallbackDemo(done) { console.log("\n--- Starting Callback Workflow ---"); const item = { id: 103, name: "Gaming Mouse", price: 50.00 }; addToCartCb(item, (err, updatedCart) => { if (err) return console.error(err); placeOrderCb(updatedCart, (err, orderDetails) => { if (err) return console.error(err); sendConfirmationEmail(orderDetails); console.log("Callback Workflow Complete!"); done(); }); }); } async function masterRunner() { await runAsyncAwaitDemo(); await runPromiseChainDemo(); runCallbackDemo(() => { console.log("\nAll demonstrations completed successfully."); }); } masterRunner(); Register No.24205038

23IT53C – Modern Web Technologies Laboratory Department Of Information Technology Output: 3.b File Upload and Processing In a document management system uploadFile(file, callback / promise / async) – uploads a file (2 sec delay), processFile(file, callback / promise / async) – processes the uploaded file(3sec delay), generateReport(file) – generates a report. Program: const uploadFile = (file) => { return new Promise((resolve) => { setTimeout(() => { console.log(`[Promise] File uploaded successfully: ${file.name}`); resolve({ ...file, status: "uploaded", serverPath: `/uploads/${file.name}` }); }, 2000); }); }; const processFile = (uploadedFile) => { return new Promise((resolve) => { setTimeout(() => { console.log(`[Promise] Finished processing file: ${uploadedFile.name}`); resolve({ ...uploadedFile, status: "processed", wordCount: 450 }); }, 3000); });}; Register No.24205038

23IT53C – Modern Web Technologies Laboratory Department Of Information Technology function

uploadFileCb(file, callback) { setTimeout(() => { console.log(`[Callback] File uploaded successfully: ${file.name}`); callback(null, { ...file, status: "uploaded", serverPath: `/uploads/${file.name}` }); }, 2000); } function processFileCb(uploadedFile, callback) { setTimeout(() => { console.log(`[Callback] Finished processing file: ${uploadedFile.name}`); callback(null, { ...uploadedFile, status: "processed", wordCount: 450 }); }, 3000); } const generateReport = (processedFile) => { console.log(`[Report] Generated report for: ${processedFile.name} (Words: ${processedFile.wordCount})`); }; async function runAsyncAwaitDemo() { console.log("\n--- Starting Async/Await File Workflow ---"); const rawFile = { name: "annual_report.pdf", size: "2.4MB" }; const uploadedFile = await uploadFile(rawFile); const processedFile = await processFile(uploadedFile); generateReport(processedFile); console.log("Async/Await File Workflow Complete!"); } function runPromiseChainDemo() { console.log("\n--- Starting Promise Chain File Workflow ---"); const rawFile = { name: "user_data.csv", size: "1.1MB" }; return uploadFile(rawFile) .then((uploadedFile) => processFile(uploadedFile)) .then((processedFile) => { generateReport(processedFile); console.log("Promise Chain File Workflow Complete!"); }); } function runCallbackDemo(done) { console.log("\n--- Starting Callback File Workflow ---"); const rawFile = { name: "invoice.docx", size: "512KB" }; uploadFileCb(rawFile, (err, uploadedFile) => { if (err) return console.error(err); processFileCb(uploadedFile, (err, processedFile) => { if (err) return console.error(err); generateReport(processedFile); console.log("Callback File Workflow Complete!"); done(); }); }); } Register No.24205038

23IT53C – Modern Web Technologies Laboratory Department Of Information Technology async function masterRunner() { await runAsyncAwaitDemo(); await runPromiseChainDemo(); runCallbackDemo(() => { console.log("\nAll document processing demonstrations completed successfully."); }); } masterRunner(); Output: Result: The given JavaScript DOM programs were successfully implemented and executed in Visual Studio Code. The expected outputs were obtained by accessing, modifying, and manipulating HTML elements using DOM methods, properties, and event handling. Problem Understanding and approach (10) Implementation and execution (15) Tool / Technology usage (5) Code Quality and Documentation (5) Viva (10) Time Management (5) Total (50) Register No.24205038